订单履约业务全景

从一笔订单的生与死，到下游事件总线的回声

gomall · 业务实战

今日大纲

一笔订单一生要走过的路

为什么下单要预扣库存而不是直接扣 DB

一笔订单的事件流

Outbox 在业务侧的角色

真实事故：库存虚高 bug 业务复盘

性能与业界对照

一笔订单一生要走过的路

为什么订单要做状态机

- 订单不是一行 DB 记录，是一段**有时间跨度的契约**：用户、商家、平台、第三方支付都在里面有义务。
- 任意一段时间内，订单只能处于**一个明确的业务态**：未支付、待发货、配送中、已收货、已取消、已退款。
- 没有状态机就没有”什么时候才能发货” / ”什么时候允许退款” / ”什么时候释放库存”的语义边界。

> 在 gomall 里，状态机 = `consts/order.go` 里那一组 iota 常量；它定义了所有合法的”现在”。

consts/order.go:3–16

gomall 的订单状态常量

Listing 1: *consts/order.go:3–16*

```go
const (
    OrderTypeUnPaid = iota + 1 // 1 未支付
    OrderTypePendingShipping // 2 已支付 待发货
    OrderTypeShipping // 3 已发货 待收货
    OrderTypeReceipt // 4 已收货 完结
)
const (
    Unknown = iota
    UnPaid // 创建时的初始态
    Paid
    Cancelled
    Refunded
)
```

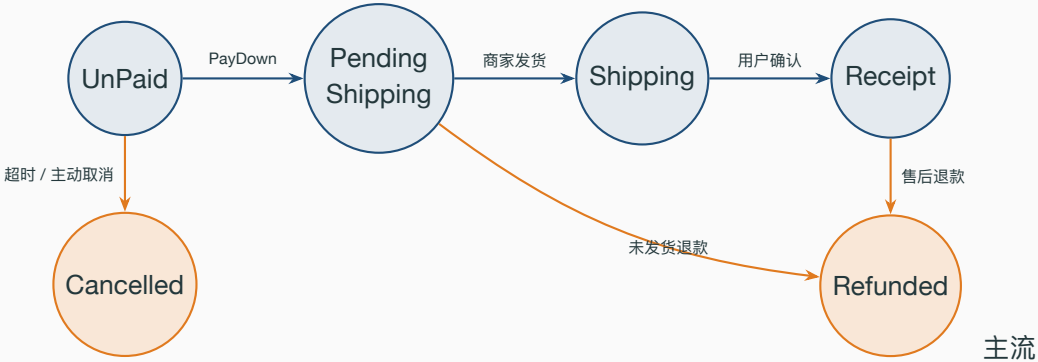
两组常量分别覆盖”主流程时序”与”取消/退款旁路”。订单创建时写入的是 `consts.UnPaid`（`service/order.go:52`）。

业务态四主线 + 两旁路

状态	含义	触发者	时长上限
UnPaid	未支付，已占住库存	用户下单	**15–30 分钟**（关单）
PendingShipping	已支付，待发货	用户付款	商家 48 小时内发货
Shipping	已发货，配送中	商家点发货	物流约 1–7 天
Receipt	已收货，交易完结	用户确认 / 系统超时	终态
Cancelled	用户取消 / 超时关单	用户 / 平台调度	任意时点（仅未支付）
Refunded	已退款	平台 / 商家审核	完结后 7–15 日内

时长上限来自电商行业惯例（淘宝 / 京东公开规则），gomall 把”未支付关单”硬编码为 15 分钟（见 `service/order.go:89` 的 zset score）和 30 分钟（见 `repository/rabbitmq/order_delay.go:22`）两套互补机制。*service/order.go:88–98 + repository/rabbitmq/order_delay.go:22*

订单状态机全景图



主流程是一条单向链；Cancelled 只能从 UnPaid 进入；Refunded 可从 PendingShipping / Receipt 进入。*service/payment.go:146 (UnPaid→PendingShipping) + service/order_cancel.go:32 (UnPaid→Cancelled)*

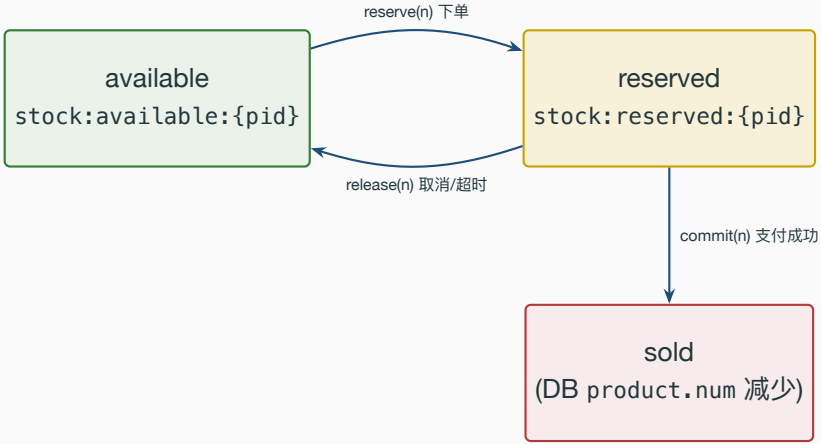
为什么下单要预扣库存而不是直接扣DB

业务问题：库存到底什么时候” 少一件”

- 直觉答案：用户下单时少一件。
- 但下单 $\neq$ 付款。用户挂着购物车不付款的比例非常高（电商业内通用统计在 60–70%）。
- 如果下单立即扣 DB `product.num`：
  - 大量未支付订单 $\rightarrow$ DB 库存被假占用 $\rightarrow$ 真正想买的用户看到” 无货”
  - 取消时必须把库存**加回去**，一旦” 取消路径” 和” 下单路径” 不对称就出 bug

stressTest/REPORT.md § 已修：库存虚高 + 历史 PR #51 / #58

gomall 的设计：两个桶



不变量：`available + reserved + sold = 初始库存`。任何一步只在两个桶间挪，不会凭空多/少。

OrderCreate 里这套桶怎么用

Listing 2: *service/order.go:57–86*

```go
// 1) Redis 预扣库存: available -> reserved, 失败直接拒单
if err = cache.ReserveStock(ctx, req.ProductID, int64(req.Num)); err != nil {
    return nil, err
}
// 2) 同事务写订单 + 写 outbox 事件
err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
    if e := dao.NewOrderDaoByDB(tx).CreateOrder(order); e != nil {
        return e
    }
    return dao.NewOutboxDaoByDB(tx).Insert(...)
})
if err != nil {
    // 3) 事务失败: 释放刚扣的预占库存, 回到 available
    cache.ReleaseReservation(ctx, req.ProductID, int64(req.Num))
}
```

三步顺序：预扣 → 写订单 → 失败回退。预扣失败直接拒单；订单写失败回退库存——状态机闭环

压测验证：500 并发抢 100 件零超卖

项	数值
初始 available 桶	100
并发 goroutine 数	500
每个 goroutine 试图 reserve	1 件
成功 reserve 数 (`success`)	**100**
available 桶最终值	**0**
reserved 桶最终值	**100**
超卖次数	**0**

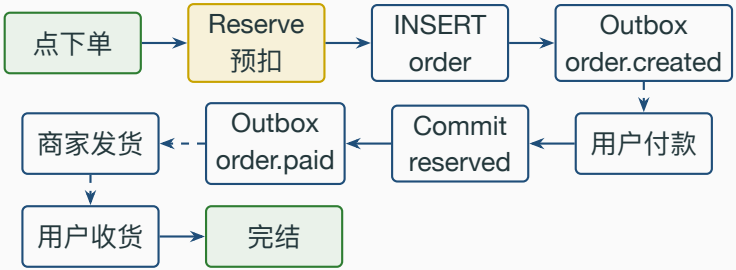
单条 Lua 脚本里 `GET+DECRBY+INCRBY` 原子执行，

500 个 goroutine 排队跑，正好抢光 100 件。*repository/cache/inventory_test.go:52–82*

`TestInventory_NoOversellUnderConcurrency`

一笔订单的事件流

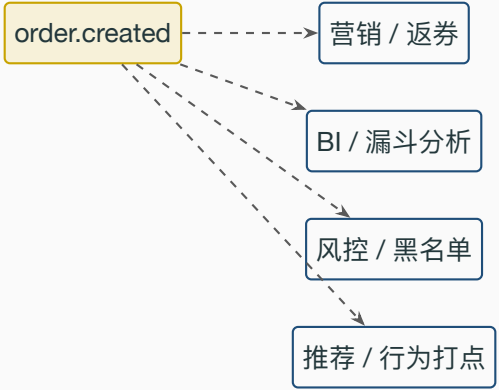
时间轴：从下单点击到完结



实线 = 同步发生在请求里；

虚线 = 异步等待（用户操作 / 物流时间）。Outbox 把异步段的开始时刻**记录下来**。

order.created 事件：谁在订阅



一个 event 多个订阅者——这正是 pub/sub 解耦的核心收益。在 gomall 现网里，`order.created` 由 publisher 投递到 `domain.events` 交换机，订阅者按 routing key 自行绑定队列。*repository/rabbitmq/domain.go:10 (`domain.events` 交换机) + service/events/events.go:9–15*

order.paid 事件：动作分裂为多条业务支线

- **库存正式扣**：`commit(reserved)` 在支付事务里同步做了；事件不再负责扣库存，但负责**通知**。
- **商家通知**：发短信 / IM，”您有新订单”。
- **推荐系统**：用户买了什么 → 用户画像更新 → 召回池调整。
- **对账系统**：把 order_id + amount 入对账明细，T+1 跟支付通道核对。
- **BI**：实时 GMV / 客单价 / 转化率。
- **营销返券**：满足规则的订单触发返券、积分、等级。

service/payment.go:174–184 (写 outbox order.paid) + service/events/events.go:17–23

order.cancelled 事件：补偿与通知

- **库存释放**：`ReleaseReservation()` 在写完 outbox 后同步执行（`service/order_cancel.go:59`）——桶状态机回退。
- **商家通知**：发”取消”提醒，让商家更新待发货列表。
- **营销回滚**：如果下单时发过预占券 / 满减锁定，要释放回去。
- **BI**：转化漏斗里这一单标记为”流失”，纳入流失分析。
- **风控**：高频取消 + 短时间内多账号 → 套券 / 刷单嫌疑。

service/order_cancel.go:30–62 + service/events/events.go:25–32

order.refunded：流程最长的一条

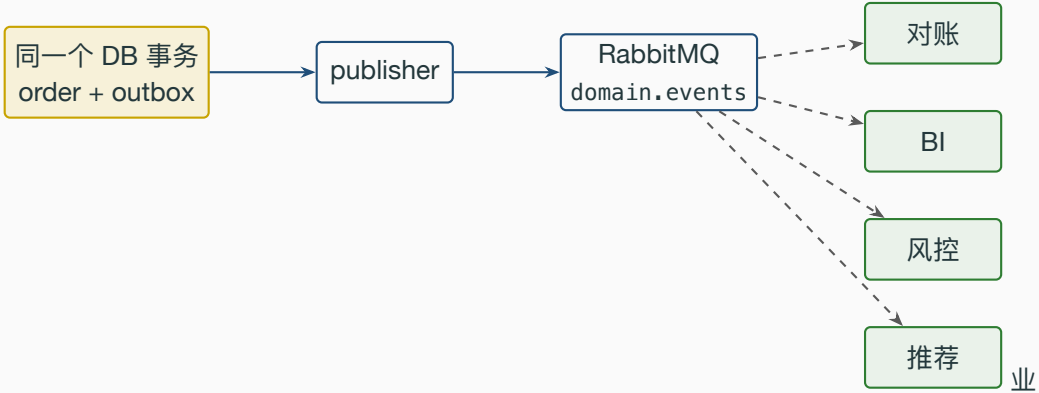
- 售后端发起 → 平台审核 → 支付通道退款 → 状态变 Refunded。
- 业务影响面：钱（账户余额）、货（要不要返仓）、人（商家信用）、税（结算回滚）。
- gomall 现在还**没实装**这一条事件链（业务态 `consts.Refunded` 是占位）；本页讲的是”如果延展到完整电商系统”的下游订阅清单。

真实电商里，refund 是一条独立 Saga：每一步都要可补偿、可重入、可对账。

consts/order.go:15 占位常量；扩展见业界 *framing* 一节

Outbox 在业务侧的角色

Outbox 把”一次业务”变成”一束事件”



业务侧不需要”逐一通知”。它只写一行 outbox，下游愿意来订就来订——**业务和增长解耦**。

repository/db/model/outbox.go:17–37

outbox_event 表结构再看一眼

Listing 3: *repository/db/model/outbox.go:24–35*

```go
type OutboxEvent struct {
    gorm.Model
    AggregateType string // "order" / "product"
    AggregateID uint // 业务实体 id
    EventType string // "OrderCreated" / "OrderCancelled"
    RoutingKey string // "order.created"
    Payload string // JSON body (events.OrderCreated 等)
    Status int // 1 pending / 2 sent / 3 dead
    Attempts int
    NextRetryAt time.Time
    LastError string
}
```

`AggregateType+AggregateID` 是业务追踪根；`RoutingKey` 决定谁能订阅；`Status` 是 publisher 的视角。

业务侧下游订阅者清单

下游	关注事件	业务价值
对账	order.paid / order.refunded	资金流闭环，T+1 跟支付通道核对
BI	全量 order.*	GMV / 客单 / 转化漏斗
风控	order.created / cancelled	异常订单识别
推荐	order.paid	行为打点 → 召回排序
营销	order.created / paid	返券 / 满减判断
商家工作台	order.paid / shipped	待办列表
ES 索引	product.changed / order.paid	搜索增量索引

真实电商每条 routing key 背后至少 3–5 个订阅者；gomall 现网先接了 ES 索引（`service/search/indexer.go`）。*service/search/indexer.go:28 (Consume) + repository/rabbitmq/domain.go:10*

真实事故：库存虚高 bug 业务复盘

现象：库存数字一路膨胀

- 商家投诉：” 我商品明明只有 100 件，怎么后台显示 130 / 160 / 200？ ”
- 售前页面显示有货，用户下单付钱，售后却” 无货发不出”。
- 客服侧不断收到” 已付款但商家说没货” 的工单。

这是一个典型的**业务表象** → **代码根因**的链条：现象在客服系统里，根因在两个 Go 函数里。

stressTest/REPORT.md § 已修：库存虚高 + service/order_task.go:33–35 `RollbackStock`

根因：一对不对称的孪生函数

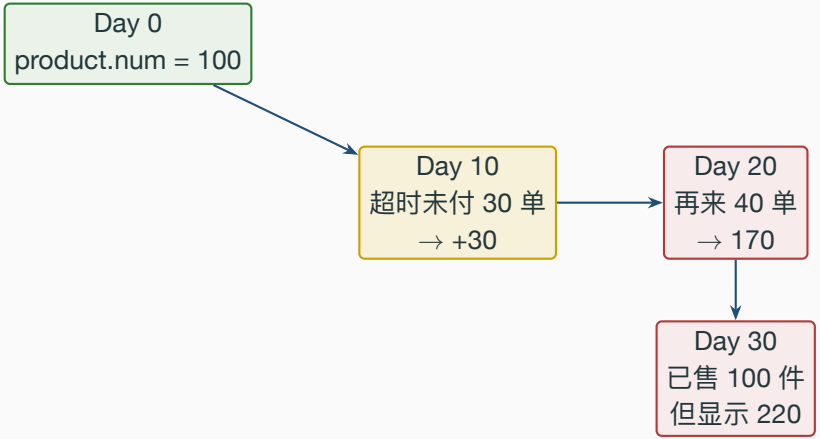
```
// 旧版 OrderCreate (PR #58 前)
// -> 没有任何 product.Num -= n 的语句
// 库存不动，只写订单

// 旧版超时关单 (service/order_task.go:33)
// txProductDao.RollbackStock(order.ProductID, order.Num)
// -> UPDATE product SET num = num + n
```

- 下单不扣，关单却”还回去”。
- 每个超时未支付订单 = 库存凭空 +N。
- 几个月累计下来，热门商品的 `product.num` 离真实值越来越远。

service/order_task.go:33 + 历史 *PR #51* 引入延迟关单时埋下伏笔

业务推演：单商品 100 件，跑一个月



” 下单不扣 + 取消加回” 这对不对称行为，配合电商常态化的 60%+ 弃单率，几乎注定会爆。

修法：把库存状态机显式化

方案	改动量	评价
A. 加一列 `reserved_num`	小	治标，状态机仍隐式
B. 双桶 + reserve/commit/release	中	**推荐**：显式三态、闭环对称

- 选 B（即 Deck 04 的双桶设计）。
- 下单 = reserve（available → reserved），不动 DB。
- 付款 = commit（reserved → 真扣 DB）。
- 取消 = release（reserved → available），**不再回写 DB**。

service/order_cancel.go:14–18 关键注释：不再回写 *product.Num*

性能与业界对照

订单列表压测：游标分页的代差碾压

指标	优化版（游标 + 缓存）	旧版（OFFSET 深分页）
吞吐 (RPS)	**58,406**	8.3
p50 延迟	1.04 ms	10.5 s
p95 延迟	**5 ms**	15.95 s
最大延迟	348 ms	16.29 s
错误率	0%	0%

同一个业务” 我的订单列表”，深分页时 p95 接近 16 秒——这在真实业务里等于直接超时；用游标 + Redis 缓存的优化版 p95 拉到 5 ms，相差**约 3,200 倍**。*stressTest/REPORT.md §2.* 订单列表：游标分页的代差碾压

业界 framing：电商订单系统怎么设计

- **阿里**：交易中心把订单建模成” 状态机 + 事件流”，公开技术博客多次提到” 扁平化状态机”+” 领域事件总线”。
- **京东**：履约链路按” 接单 / 履约 / 完结” 三大域，每个域独立状态机，通过事件解耦。QCon 公开分享强调” 补偿事务 + 异步消息” 是底线设计。
- **Saga 模式**：业界电商对长事务的事实标准——gomall 用 Outbox + 协同式 Saga 是同一族思路。

维度	gomall	大型电商
状态数	4 主 + 2 旁	数十个细分
事件总线	单 RabbitMQ exchange	多级 / 跨域 / 多机房
补偿事务	协同式 Saga + 释放库存	编排式 Saga + 人工兜底

原理一致，工程量天差地别。这里只列方向不引数字，避免不可考的” 百万 TPS” 之类。

面试 Q&A（上）

Q1：为什么下单不直接扣 DB?

A：DB 库存 ≠ 售前可见库存。下单只是占用，付款才是消耗。两者混淆会导致” 超时单 + 反向补偿” 的不对称 bug。预扣（reserved 桶）就是把” 占用” 这个时间窗口显式建模。

Q2：reserved 桶里的库存，售前要怎么显示?

A：售前页面读 `available` 桶——reserved 对售前不可见。

`available + reserved + sold =` 初始水位这个不变量保证你怎么算都不会超。

Q3：为什么不直接 DB 加锁 `SELECT FOR UPDATE`?

A：可以，但每次下单都锁行 + 等锁排队，吞吐会塌。Redis Lua 是**单线程原子**，性能差一个数量级。本项目压测 500 并发零超卖（`repository/cache/inventory_test.go:52--82`）。

面试 Q&A（下）

Q4：order.paid 事件的下游有哪些？

A：对账、商家通知、推荐、BI、营销返券、ES 索引。Outbox + RabbitMQ 把它们都解耦——业务侧只写一行 `outbox.Insert`。

Q5：库存虚高事故你怎么排查？

A：从业务现象（商家投诉、售前售后不一致）反推：库存只增不减 → 找所有 `product.Num += ` 的地方 → 发现 `RollbackStock`，再找有没有对称的 `-=` → 发现 OrderCreate 根本没扣 → 根因是状态机隐式 / 路径不对称。修法：显式化双桶状态机。

反追问：如果未付款订单有几十万，关单怎么扛？

A：本项目用 RabbitMQ 死信队列做延迟关单（`repository/rabbitmq/order_delay.go:22` 默认 30 分钟）+ 定时任务兜底（`service/order_task.go` 扫超过 15 分钟的）。两条路径都做了幂等（`CloseOrderWithCheck`），重复触发 no-op。

代码位置一览

订单状态常量 *consts/order.go:3–16*

OrderCreate + 预扣 + Outbox *service/order.go:40–102*

PayDown + 状态转移 + Commit *service/payment.go:35–200*

CancelUnpaidOrder + Release *service/order_cancel.go:14–63*

双桶库存状态机 *repository/cache/inventory.go:16–138*

Outbox 表 + 事件类型 *repository/db/model/outbox.go:17–37*

事件 payload *service/events/events.go:9–38*

延迟关单拓扑 *repository/rabbitmq/order_delay.go:13–105*

500 并发库存测试 *repository/cache/inventory_test.go:52–82*

> 业务复盘的关键引用：`service/order_cancel.go:14--18` 注释里写明”不再回写 product.Num”。